



Lecture 6 Logistic Regression and Linear Classifier

大数据分析 [2026 国际商务] · Big Data Analytics

任课教师: 戴谢尔 (Xieer Dai, David)

深圳大学 Shenzhen University · 第 3-14 周·星期四 11-12 节·四方楼南 209

1. Linear Regression

从回归到分类的第一步：线性模型与最大似然

Example 1: image classification

- ▶ **Task:** determine whether an image is indoor or outdoor;
- ▶ **Performance measure:** probability of misclassification;
- ▶ **Experience / data:** images together with labels (indoor/outdoor);
- ▶ **Learning:** a binary classifier $f(\mathbf{x}) \in \{0, 1\}$.

Binary classification

Classify each input image into one of two classes — the simplest supervised learning problem; later we generalise it to multi-class (k classes).





深圳大学
SHENZHEN UNIVERSITY

Example 1: image classification

Experience/Data:
images with labels



Indoor
深圳大学
SHENZHEN UNIVERSITY

outdoor

Big Data Analytics in Business

Example 1: image classification

A few terminologies

- ▶ **Training data:** the images given for learning;
- ▶ **Test data:** the images to be classified (unseen during learning);
- ▶ **Binary classification:** classify into two classes.

Supervised learning uses labelled examples (training data) to learn a rule that predicts the label of unseen examples (test data).



深圳大学
SHENZHEN UNIVERSITY

Example 1: image classification (multi-class)



深圳大学
SHENZHEN UNIVERSITY

ImageNet figure borrowed from vision.stanford.edu
Big Data Analytics in Business



Example 2: clustering images



Task: partition the images into 2 groups
Performance: similarities within groups
Data: a set of images



Example 2: clustering images

A few terminologies

- ▶ **Unlabeled data** vs. **labeled data**;
- ▶ **Supervised learning** vs. **unsupervised learning**: clustering is **unsupervised** — only $\{\mathbf{x}_i\}$ are observed, no labels y_i .

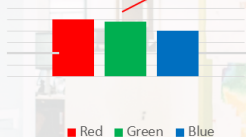
Clustering: partition the images into k groups so that images within one group are similar (Task). Performance is measured by within-group similarity; the data are just a set of images (Experience).

Math formulation



Extract
features

Color Histogram



Feature vector: x_i

Label: y_i

Indoor

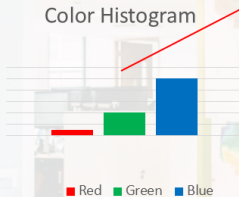
0



Math formulation



Extract
features



Feature vector: x_j

outdoor

1

Label: y_j



Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$:

- ▶ **Find** a function $y = f(\mathbf{x})$ *using the training data*;
- ▶ **s.t.** f is correct on the *test data* (examples not seen in training).

What kind of functions f can we use?

Choosing the *function class* (hypothesis class) is the first modelling decision.

Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$:

- ▶ **Find** $y = f(\mathbf{x}) \in \mathcal{H}$ using the training data;
- ▶ **s.t.** f is correct on the test data.

Hypothesis class $\mathcal{H}$

We restrict the search to a hypothesis class $\mathcal{H}$ (e.g. all linear functions $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$). The class encodes our prior beliefs about the task.

Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$:

- ▶ **Find** $y = f(\mathbf{x}) \in \mathcal{H}$ using the training data;
- ▶ **s.t.** f is correct on the test data.

Connection between training data and test data?

If the test data were unrelated to the training data, no rule learned from training could generalise. A standard assumption makes the connection precise.

Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ *i.i.d.* from distribution $\mathcal{D}$:

- ▶ **Find** $y = f(\mathbf{x}) \in \mathcal{H}$ using the training data;
- ▶ **s.t.** f is correct on test data, which are *also* *i.i.d.* from $\mathcal{D}$.

They have the same distribution

i.i.d.: *independently identically distributed* — training and test points are drawn independently from the same unknown distribution $\mathcal{D}$; this is what makes generalisation from training to test possible.

Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$:

- ▶ **Find** $y = f(\mathbf{x}) \in \mathcal{H}$ using the training data;
- ▶ **s.t.** f is correct on test data i.i.d. from $\mathcal{D}$.

What kind of performance measure?

To *choose* f we need a quantitative measure of how good a function is: a loss function $\ell(f, \mathbf{x}, y)$ and its expectation over $\mathcal{D}$.

Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$:

- ▶ **Find** $y = f(\mathbf{x}) \in \mathcal{H}$ using the training data;
- ▶ **s.t.** the **expected loss** is small.

Various loss functions $\ell(f, \mathbf{x}, y)$

Expected loss (risk)

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\ell(f, \mathbf{x}, y)]$$

Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$; require the expected loss $\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}}[\ell(f, \mathbf{x}, y)]$ to be small.

Examples of loss functions

0-1 loss (classification)

$$\ell(f, \mathbf{x}, y) = \mathbb{I}[f(\mathbf{x}) \neq y], \quad L_{\mathcal{D}}(f) = \Pr_{(\mathbf{x}, y) \sim \mathcal{D}}[f(\mathbf{x}) \neq y]$$

ℓ_2 loss (regression)

$$\ell(f, \mathbf{x}, y) = [f(\mathbf{x}) - y]^2, \quad L_{\mathcal{D}}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}}[(f(\mathbf{x}) - y)^2]$$

Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$:

- ▶ **Find** $y = f(\mathbf{x}) \in \mathcal{H}$ using the training data;
- ▶ **s.t.** the expected loss $L_{\mathcal{D}}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}}[\ell(f, \mathbf{x}, y)]$ is small.

How to use it?

The expected loss involves the *unknown* distribution $\mathcal{D}$, so it cannot be computed. We replace it by the *empirical loss* on the training data (next slide) — this is the statistical learning principle “empirical risk minimisation”.

Math formulation

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$:

- **Find** $y = f(\mathbf{x}) \in \mathcal{H}$ that **minimises the empirical loss**

$$\hat{L}(f) = \frac{1}{n} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i),$$

- **s.t.** the expected loss $L_{\mathcal{D}}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}}[\ell(f, \mathbf{x}, y)]$ is small.

Empirical loss $\hat{L}(f)$

The empirical loss is the Monte-Carlo estimate of the expected loss using the n training examples: as $n \rightarrow \infty$, $\hat{L}(f) \rightarrow L_{\mathcal{D}}(f)$.

Machine learning 1-2-3

The whole procedure reduces to three steps:

1. **Collect data and extract features;**
2. **Build model:** choose a hypothesis class $\mathcal{H}$ and a loss function ℓ ;
3. **Optimization:** minimise the empirical loss over $\mathcal{H}$.

Steps 2–3 formalise *learning* as an optimisation problem:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i).$$

Wait...

- ▶ Why *handcraft* the feature vectors $\mathbf{x}, y$? We can use prior knowledge to design suitable features;
- ▶ Can the computer learn the features on the raw images?
 - Learning features directly from raw data is called **Representation Learning**;

The AI / ML landscape

Deep Learning $\subseteq$ Representation Learning $\subseteq$ Machine Learning $\subseteq$ Artificial Intelligence

Wait...

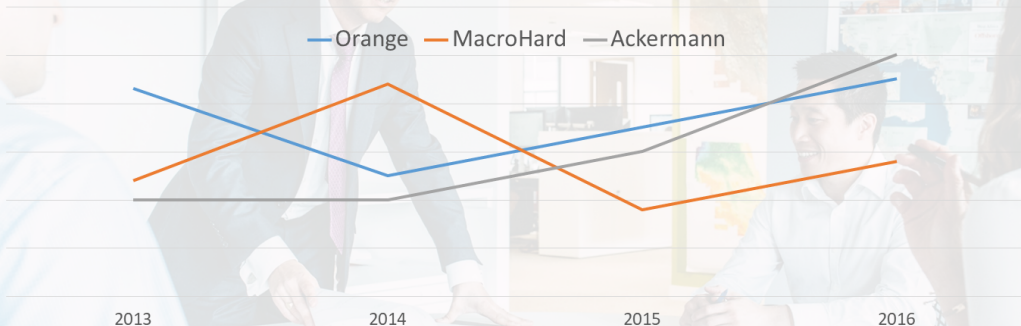
- ▶ Does “Machine Learning 1-2-3” include *all* approaches?
 - It includes many, but not all (e.g. not pure rule-based or instance-based reasoning without an explicit loss);
- ▶ Our current focus will be on **Machine Learning 1-2-3**.

This course follows the “collect data → build model → optimise” framework; everything else is treated as background.



Example: Stock Market Prediction

Stock Market (Disclaimer: synthetic data/in another parallel universe)



Sliding window over time: serve as input x ; non-i.i.d.

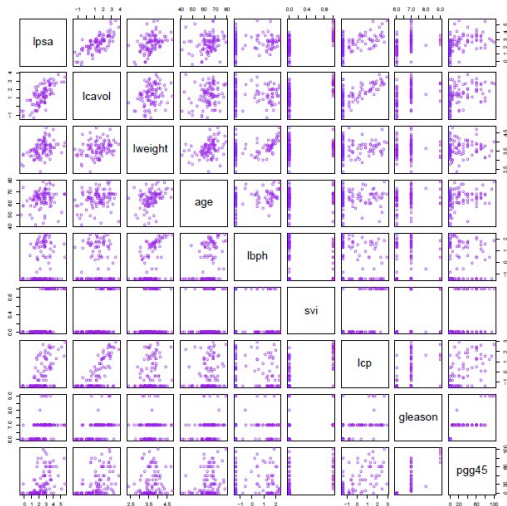
Big Data Analytics in Business



Linear regression

$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$: 最小二乘与闭式解

Linear regression: example data



- ▶ y : prostate specific antigen (log-scale), the outcome to predict;
- ▶ $(x_1, \dots, x_8)$: clinical measures used as features;
- ▶ Regression of y on x fits the linear relation

$$y \approx \mathbf{w}^T \mathbf{x}.$$

Real data: Prostate Cancer by Stamey et al. (1989). Figure borrowed from *The Elements of Statistical Learning*.

Linear regression

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$, $y \in \mathbb{R}$.

► **Hypothesis class $\mathcal{H}$ (linear model):**

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}.$$

► **Loss:** ℓ_2 loss (also called *mean square error*);

Find $f_{\mathbf{w}}$ that minimises the empirical loss

$$\hat{L}(f_{\mathbf{w}}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2.$$

Choosing the loss = choosing what “good” means; the ℓ_2 loss is smooth and leads to a closed-form solution (below).

Linear regression: optimization

- Objective (empirical loss, ℓ_2):

$$\hat{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2.$$

- **Matrix form.** Let $\mathbf{X}$ be the $n \times d$ matrix whose i -th row is $\mathbf{x}_i^\top$ and let $\mathbf{y} = (y_1, \dots, y_n)^\top$; then

$$\hat{L}(\mathbf{w}) = \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{n} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}).$$

Goal: minimise $\hat{L}(\mathbf{w})$ over $\mathbf{w}$

Linear regression: optimization

Set the gradient to zero to obtain the minimiser (convex quadratic):

$$\nabla_{\mathbf{w}} \hat{L}(\mathbf{w}) = \frac{1}{n} \nabla_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = 0$$

$$\iff \nabla_{\mathbf{w}} \left[(\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \right] = 0$$

$$\iff \nabla_{\mathbf{w}} \left[\mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y} \right] = 0$$

$$\iff 2\mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{X}^\top \mathbf{y} = 0$$

Normal equation

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Linear regression: optimization

Algebraic view of the minimiser

- ▶ If $\mathbf{X}$ were (square and) invertible, we could just solve $\mathbf{X}\mathbf{w} = \mathbf{y}$ and get $\mathbf{w} = \mathbf{X}^{-1}\mathbf{y}$;
- ▶ But typically $\mathbf{X}$ is a **tall** matrix (n rows $\gg$ d columns): more equations than unknowns — no exact solution in general.

Normal equation (least squares)

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y} \quad \Longleftrightarrow \quad \mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Minimises the squared residuals $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ over all $\mathbf{w}$.

Linear regression with bias

- ▶ Model with an intercept (bias term b):

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b.$$

- ▶ **Reduce to the case without bias** (augmentation trick): define

$$\mathbf{w}' = \begin{pmatrix} \mathbf{w} \\ b \end{pmatrix}, \quad \mathbf{x}' = \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix},$$

then

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = (\mathbf{w}')^\top \mathbf{x}'.$$

Augmenting every feature vector with a constant 1 absorbs the bias into the weight vector, so all earlier formulas (incl. the normal equation) apply unchanged.

Math formulation (recap)

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$:

- **Find** $y = f(\mathbf{x}) \in \mathcal{H}$ minimising the **empirical loss**

$$\hat{L}(f) = \frac{1}{n} \sum_{i=1}^n \ell(f, \mathbf{x}_i, y_i),$$

- **s.t.** the expected loss $L_{\mathcal{D}}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\ell(f, \mathbf{x}, y)]$ is small (the quantity we truly care about).

Linear regression instantiates this scheme with $\mathcal{H} = \{f_{\mathbf{w}} = \mathbf{w}^{\top} \mathbf{x}\}$ and $\ell =$ squared error; MLE (next) tells us how to *derive* losses.

Machine learning 1-2-3

1. **Collect data and extract features;**
2. **Build model:** choose a hypothesis class $\mathcal{H}$ and a loss function ℓ ;
3. **Optimization:** minimise the empirical loss.

Machine learning 1-2-3

1. **Collect data and extract features;** (Experience)
 2. **Build model:** choose hypothesis class $\mathcal{H}$ and loss ℓ ;
(Prior knowledge)
 3. **Optimization:** minimise the empirical loss.
- ▶ **Experience** provides the data (and the task);
 - ▶ **Prior knowledge** selects the model class and the loss;
 - ▶ **Optimization** turns data+model into a concrete predictor.

Example: Linear regression

Setup. Training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$.

► **Find** $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ that minimises the empirical loss

$$\hat{L}(f_{\mathbf{w}}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2;$$

► **Loss:** ℓ_2 loss; **Hypothesis class:** linear model $\mathcal{H}$.

Already solved: $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ — closed form, no iterative optimisation needed.

Why ℓ_2 loss

- ▶ **Why not choose another loss?** e.g. ℓ_1 loss, hinge loss, exponential loss, ...
- ▶ **Empirical reason:** easy to optimize. For the linear case there is a closed-form minimiser

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

- ▶ **Theoretical reason:** a way to encode **prior knowledge** about the noise model.

Questions

- ▶ What kind of prior knowledge?
- ▶ Is there a *principal* way to derive the loss function?

These questions motivate **Maximum Likelihood Estimation** (next part).

Maximum likelihood

从“先验”导出损失函数：极大似然估计 MLE

Maximum likelihood Estimation (MLE)

- ▶ Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from distribution $\mathcal{D}$;
- ▶ Let $\{P_\theta(\mathbf{x}, y) : \theta \in \Theta\}$ be a family of (joint) distributions on $(\mathbf{x}, y)$, indexed by the parameter θ ;
- ▶ Would like to pick θ so that $P_\theta(\mathbf{x}, y)$ “fits the data well”.

Key idea: choose the distribution (parameter) under which the observed data are *most probable*.

Maximum likelihood Estimation (MLE)

- ▶ Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$;
- ▶ Let $\{P_\theta(\mathbf{x}, y) : \theta \in \Theta\}$ be a parametric family of distributions;
- ▶ **“Fitness” of θ to one data point $(\mathbf{x}_i, y_i)$ — the likelihood:**

$$\text{likelihood}(\theta; \mathbf{x}_i, y_i) := P_\theta(\mathbf{x}_i, y_i).$$

Maximum likelihood Estimation (MLE)

- ▶ Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$;
- ▶ Let $\{P_\theta(\mathbf{x}, y) : \theta \in \Theta\}$ be a parametric family of distributions;
- ▶ **“Fitness” of θ to the whole i.i.d. sample** — joint likelihood factorises into a product:

$$\text{likelihood}(\theta; \{(\mathbf{x}_i, y_i)\}) := P_\theta(\{(\mathbf{x}_i, y_i)\}) = \prod_{i=1}^n P_\theta(\mathbf{x}_i, y_i).$$

Maximum likelihood Estimation (MLE)

MLE: maximise the “fitness” of θ to the i.i.d. data points $\{(\mathbf{x}_i, y_i)\}$:

$$\theta_{\text{ML}} = \arg \max_{\theta \in \Theta} \prod_{i=1}^n P_{\theta}(\mathbf{x}_i, y_i).$$

Maximising a product of probabilities is numerically awkward (underflow for large n); take the logarithm instead — monotone, so the maximiser is unchanged.

Maximum likelihood Estimation (MLE)

MLE: maximise the log of the product:

$$\theta_{\text{ML}} = \arg \max_{\theta \in \Theta} \log \left[\prod_{i=1}^n P_{\theta}(\mathbf{x}_i, y_i) \right] = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log [P_{\theta}(\mathbf{x}_i, y_i)].$$

The logarithm turns the product into a sum (log-likelihood), which is numerically stable and analytically convenient.

Maximum likelihood Estimation (MLE)

Maximising $\sum_i \log P_\theta(\mathbf{x}_i, y_i)$ is equivalent to minimising its negative:

Negative log-likelihood loss

$$\ell(P_\theta, \mathbf{x}_i, y_i) = -\log(P_\theta(\mathbf{x}_i, y_i)), \quad \hat{L}(P_\theta) = -\sum_{i=1}^n \log(P_\theta(\mathbf{x}_i, y_i)).$$

$$\theta_{\text{ML}} = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log(P_\theta(\mathbf{x}_i, y_i)) = \arg \min_{\theta \in \Theta} \hat{L}(P_\theta).$$

MLE: conditional log-likelihood

Change of view. Model only the **conditional** distribution of the label given the features:

$$\{P_{\theta}(y \mid \mathbf{x}) : \theta \in \Theta\}.$$

- **MLE:** maximise the conditional log-likelihood

$$\theta_{\text{ML}} = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log (P_{\theta}(y_i \mid \mathbf{x}_i));$$

- **Equivalent loss** (negative conditional log-likelihood):

$$\ell(P_{\theta}, \mathbf{x}_i, y_i) = -\log (P_{\theta}(y_i \mid \mathbf{x}_i)), \quad \widehat{L}(P_{\theta}) = -\sum_{i=1}^n \log (P_{\theta}(y_i \mid \mathbf{x}_i)).$$

Only care about predicting y from $\mathbf{x}$; we do not care about $P(\mathbf{x})$.

MLE: conditional log-likelihood

Modelling the conditional $P_{\theta}(y \mid \mathbf{x})$ with the negative conditional log-likelihood loss:

$$\theta_{\text{ML}} = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log P_{\theta}(y_i \mid \mathbf{x}_i), \quad \hat{L}(P_{\theta}) = - \sum_{i=1}^n \log P_{\theta}(y_i \mid \mathbf{x}_i).$$

$P(y \mid \mathbf{x})$: **discriminative**; $P(\mathbf{x}, y)$: **generative**

Discriminative models estimate $P(y \mid \mathbf{x})$ directly (only what is needed for prediction); **generative** models estimate the full joint $P(\mathbf{x}, y)$ (can also sample/synthesise new $\mathbf{x}$).

Example: ℓ_2 loss

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$. **Find** a function $f_\theta(\mathbf{x})$ (parameters θ) that minimises

$$\hat{L}(f_\theta) = \frac{1}{n} \sum_{i=1}^n (f_\theta(\mathbf{x}_i) - y_i)^2.$$

Where does this ℓ_2 loss come from? Answer: MLE under Gaussian noise (next slide).

Example: ℓ_2 loss $\Leftrightarrow$ Normal noise + MLE

Assume the label is the function value plus Gaussian noise:

$$P_{\theta}(y | \mathbf{x}) = \mathcal{N}(y; f_{\theta}(\mathbf{x}), \sigma^2).$$

Then the conditional log-likelihood of one point is

$$\log P_{\theta}(y_i | \mathbf{x}_i) = -\frac{1}{2\sigma^2} (f_{\theta}(\mathbf{x}_i) - y_i)^2 - \log \sigma - \frac{1}{2} \log(2\pi).$$

MLE $\Rightarrow$ least squares

$$\theta_{\text{ML}} = \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n (f_{\theta}(\mathbf{x}_i) - y_i)^2$$

(σ and the constants do not affect the argmin).

Linear classification

从 step 函数到 Logistic 回归



深圳大学
SHENZHEN UNIVERSITY

Example 1: image classification



indoor



outdoor



深圳大学
SHENZHEN UNIVERSITY

Big Data Analytics in Business

Example 2: Spam detection

Features: counts of suspicious keywords; **label:** spam or not.

Email	#"\$"	#"Mr."	#"sale"	...	Spam?
Email 1	2	1	1		Yes
Email 2	0	1	0		No
Email 3	1	1	1		Yes
...					
Email n	0	0	0		No
New email	0	0	1		??

Each email is encoded by a feature vector $\mathbf{x}$ (word counts). The row “New email” has to be *predicted* by the classifier trained on the first n labelled emails — a binary classification task.

Why classification

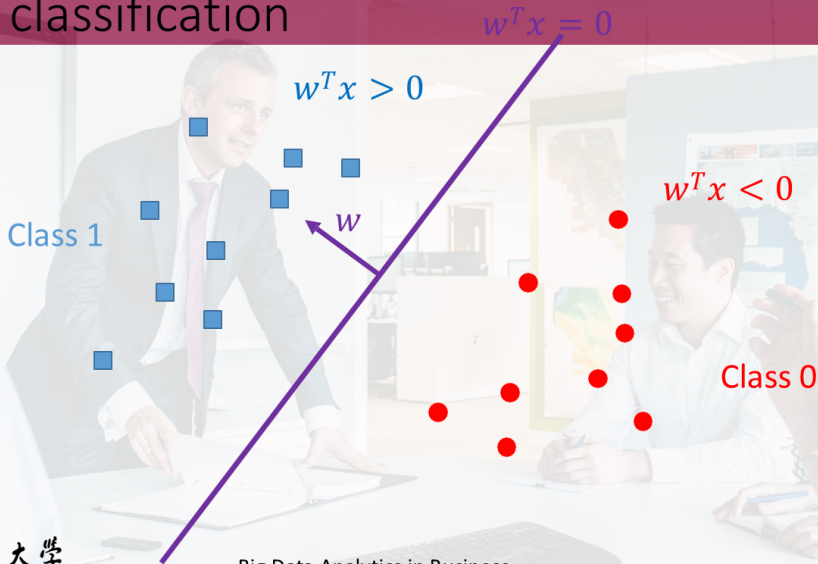
- ▶ Classification is a kind of **summary**: many raw signals collapse into a small set of classes;
- ▶ Easy to **interpret** (a “yes/no” answer for a decision);
- ▶ Easy for **making decisions** (each class triggers an action).

Examples: spam/not-spam, indoor/outdoor, fraud/not-fraud, buy/hold/sell, default/not-default ...



深圳大学
SHENZHEN UNIVERSITY

Linear classification



深圳大学
SHENZHEN UNIVERSITY

Big Data Analytics in Business

Linear classification: natural attempt

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$, with $y_i \in \{0, 1\}$.

► **Hypothesis (linear model $\mathcal{H}$):**

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x},$$

► **Prediction via a step function:**

$$\hat{y} = \text{step}(f_{\mathbf{w}}(\mathbf{x})) = \text{step}(\mathbf{w}^\top \mathbf{x}), \quad \hat{y} = \begin{cases} 1, & \mathbf{w}^\top \mathbf{x} > 0, \\ 0, & \mathbf{w}^\top \mathbf{x} < 0. \end{cases}$$

$\mathbf{w}^\top \mathbf{x} = 0$ is the decision boundary (a hyperplane through the origin)

Linear classification: natural attempt

Find $\mathbf{w}$ that minimises the empirical 0-1 loss:

$$\hat{L}(f_{\mathbf{w}}) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[\text{step}(\mathbf{w}^\top \mathbf{x}_i) \neq y_i].$$

- **Drawback:** the 0-1 loss is *difficult to optimize* (discontinuous, flat almost everywhere, non-convex);
- It is **NP-hard** in the worst case.

Consequence: replace the 0-1 loss by a smooth surrogate that is easy to optimise — first the ℓ_2 loss (linear regression view), then the log-loss / logistic loss.

Linear classification: simple approach

Setup. Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$, $y_i \in \{0, 1\}$. **Find** $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ that minimises

$$\hat{L}(f_{\mathbf{w}}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2.$$

Reduce to linear regression; ignore the fact that $y \in \{0, 1\}$

- ▶ The closed-form least-squares solution applies;
- ▶ But the linear fit is **not robust to outliers** (next slide) and can predict outside $[0, 1]$.

Linear classification: simple approach

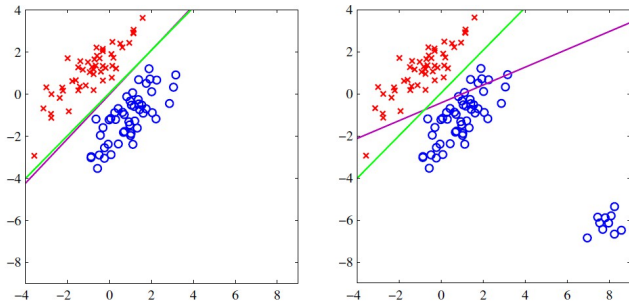


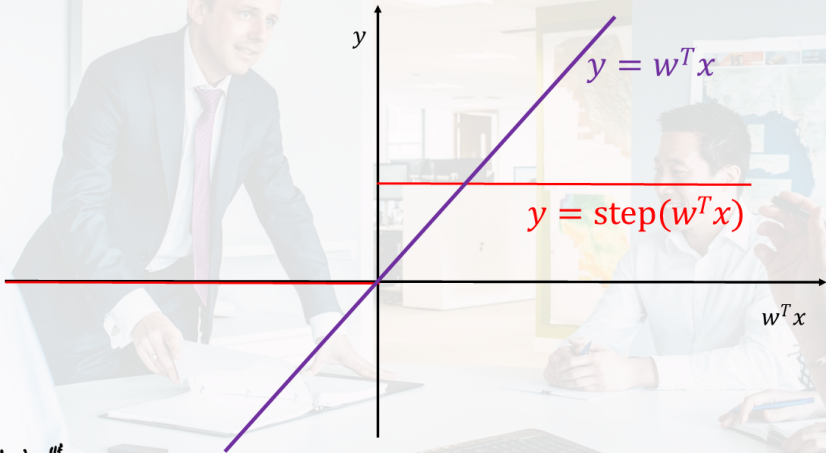
Figure 4.4 The left plot shows data from two classes, denoted by red crosses and blue circles, together with the decision boundary found by least squares (magenta curve) and also by the logistic regression model (green curve), which is discussed later in Section 4.3.2. The right-hand plot shows the corresponding results obtained when extra data points are added at the bottom left of the diagram, showing that least squares is highly sensitive to outliers, unlike logistic regression.

- ▶ Regression view is **not robust to outliers**: one distant point can shift the fitted line $\mathbf{w}^\top \mathbf{x} = 0$ so that the boundary misclassifies many inliers;
- ▶ Least squares responds to the *magnitude* of the residuals, although for classification only the *sign* matters.

Figure borrowed from *Pattern Recognition and Machine Learning*, Bishop (2006).



Compare the two



Between the two: sigmoid

- ▶ We want a smooth function whose output is **bounded in $[0, 1]$** ;
- ▶ The **sigmoid** (logistic) function does exactly this:

$$\sigma(a) = \frac{1}{1 + \exp(-a)} \in (0, 1).$$

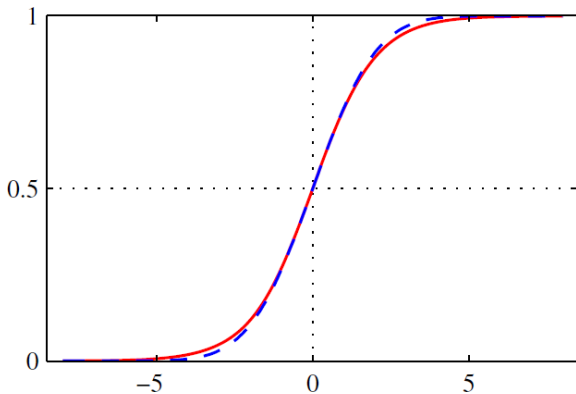


Figure borrowed from *Pattern Recognition and Machine Learning*, Bishop (2006).

Linear classification: sigmoid prediction

Squash the output of the linear function into $(0, 1)$:

$$\text{Sigmoid}(\mathbf{w}^\top \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}.$$

One possible approach: least squares on the squashed output — **find $\mathbf{w}$** minimising

$$\hat{L}(f_{\mathbf{w}}) = \frac{1}{n} \sum_{i=1}^n \left(\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i \right)^2.$$

Better idea (next slide): treat the squashed output *as a probability* and use MLE instead of least squares.

Linear classification: logistic regression

A better approach: interpret the output as a probability.

Probabilistic linear classifier

$$P_{\mathbf{w}}(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})},$$
$$P_{\mathbf{w}}(y = 0 \mid \mathbf{x}) = 1 - P_{\mathbf{w}}(y = 1 \mid \mathbf{x}) = 1 - \sigma(\mathbf{w}^\top \mathbf{x}).$$

Prediction rule: $\hat{y} = 1$ iff $\sigma(\mathbf{w}^\top \mathbf{x}) > 0.5$, i.e. iff $\mathbf{w}^\top \mathbf{x} > 0$ — the same hyperplane boundary as before.

Linear classification: logistic regression

Given $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ i.i.d. from $\mathcal{D}$, $y_i \in \{0, 1\}$, fit the model by **conditional MLE**
 $P_{\mathbf{w}}(y | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$.

Negative conditional log-likelihood loss

$$\hat{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i: y_i=1} \log \sigma(\mathbf{w}^\top \mathbf{x}_i) - \frac{1}{n} \sum_{i: y_i=0} \log (1 - \sigma(\mathbf{w}^\top \mathbf{x}_i))$$

Logistic regression: MLE with sigmoid (= cross-entropy loss)

Linear classification: logistic regression

- ▶ Objective (negative conditional log-likelihood / cross-entropy):

$$\hat{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i: y_i=1} \log \sigma(\mathbf{w}^\top \mathbf{x}_i) - \frac{1}{n} \sum_{i: y_i=0} \log (1 - \sigma(\mathbf{w}^\top \mathbf{x}_i)).$$

- ▶ Unlike least squares, there is **no closed-form solution**;
- ▶ **Need to use gradient descent** (or another iterative optimiser).

The loss is convex in $\mathbf{w}$, so gradient descent converges to the unique global minimum.

Properties of sigmoid function

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

► **Bounded:**

$$\sigma(a) = \frac{1}{1 + e^{-a}} \in (0, 1) \quad \forall a \in \mathbb{R};$$

► **Symmetric:**

$$1 - \sigma(a) = \frac{e^{-a}}{1 + e^{-a}} = \frac{1}{e^a + 1} = \sigma(-a);$$

► **Gradient (key for learning):**

$$\sigma'(a) = \frac{e^{-a}}{(1 + e^{-a})^2} = \sigma(a)(1 - \sigma(a)).$$

The identity $\sigma' = \sigma(1 - \sigma)$ makes the gradient of the logistic loss remarkably simple to compute.

3.1 Penalized Regression

Idea. Penalized regression constrains the regression coefficients so that they minimise the sum of squared residuals *plus* a penalty term that increases with the number / size of the included features.

- ▶ A feature must make a **sufficient contribution** to model fit to offset the penalty for including it;
- ▶ Hence only the more important features for explaining y remain in the final penalized regression model.

Generic penalized least squares

$$\hat{\beta} = \arg \min_{\beta} \underbrace{\sum_{i=1}^n (y_i - \beta^\top \mathbf{x}_i)^2}_{\text{fit (RSS)}} + \lambda \cdot \underbrace{\text{pen}(\beta)}_{\text{penalty}}, \quad \lambda > 0$$

3.1 Penalized Regression: three methods

Methods covered by the lecture:

- ▶ **LASSO** regression — penalty $\propto \sum_j |\beta_j|$ (ℓ_1); performs feature selection (sparse solution);
- ▶ **Ridge** regression — penalty $\propto \sum_j \beta_j^2$ (ℓ_2); shrinks coefficients but keeps all features;
- ▶ **Elastic-Net** — combination of the two penalties.

Penalty terms ($\lambda > 0$)

$$\text{LASSO} : \lambda \sum_{j=1}^d |\beta_j|, \quad \text{Ridge} : \lambda \sum_{j=1}^d \beta_j^2, \quad \text{Elastic Net} : \lambda_1 \sum_{j=1}^d |\beta_j| + \lambda_2 \sum_{j=1}^d \beta_j^2$$

Prediction problem set-up

Given:

- ▶ Training data set $\{(\mathbf{y}_i, \mathbf{x}_i)\}_{i=1}^n$ (assume i.i.d.);
- ▶ Usually called **regression** when y is continuous, and **classification** when y is discrete;
- ▶ A *loss function* $\ell(y, f(\mathbf{x}))$.

Goal:

- ▶ A prediction function f with **low average loss** (“risk”)

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{y}, \mathbf{x}) \sim \mathcal{D}} [\ell(\mathbf{y}, f(\mathbf{x}))],$$

where $(\mathbf{y}, \mathbf{x})$ are distributed as the training data.

Econometrics vs. ML vocabulary

	Econometrics	ML
y	Outcome variable	Label
x	Covariate	Feature

Squared-error loss for regression

“Regression”: continuous outcome, $y \in \mathbb{R}$.

Squared-error loss

$$\ell(y, f(\mathbf{x})) = (y - f(\mathbf{x}))^2, \quad \mathcal{R}(f) = \mathbb{E}_{(\mathbf{y}, \mathbf{x}) \sim \mathcal{D}} \left[(\mathbf{y} - f(\mathbf{x}))^2 \right].$$

Examples discussed in the literature:

- ▶ Predict log house price y of a new home from its characteristics $\mathbf{x}$ based on survey data from homes with the same distribution (*Mullainathan and Spiess, 2017*);
- ▶ Predict log consumption y for a new household $\mathbf{x}$ based on data on similar households (*Adelman et al.*).

Squared-error loss for regression

Performance of Different Algorithms in Predicting House Values

Method	Prediction performance (R^2)		Relative improvement over ordinary least squares by quintile of house value				
	Training sample	Hold-out sample	1st	2nd	3rd	4th	5th
Ordinary least squares	47.3%	41.7% [39.7%, 43.7%]	–	–	–	–	–
Regression tree tuned by depth	39.6%	34.5% [32.6%, 36.5%]	–11.5%	10.8%	6.4%	–14.6%	–31.8%
LASSO	46.0%	43.3% [41.5%, 45.2%]	1.3%	11.9%	13.1%	10.1%	–1.9%
Random forest	85.1%	45.5% [43.6%, 47.5%]	3.5%	23.6%	27.0%	17.8%	–0.5%
Ensemble	80.4%	45.9% [44.0%, 47.9%]	4.5%	16.0%	17.9%	14.2%	7.6%

Benchmark: the dependent variable is the log-dollar house value of owner-occupied units in the 2011 American Housing Survey, using 150 covariates (unit characteristics and quality measures). All algorithms are fitted on the same randomly drawn training sample of 10,000 units and evaluated on the 41,808 remaining held-out units. Numbers in brackets are 95% bootstrap confidence intervals for hold-out prediction performance (measurement variation for a fixed prediction function). No sampling weights are used. Details: online Appendix at <http://e-jep.org>.

Loss measures for classification

“**Classification**”: binary outcome, $y \in \{0, 1\}$.

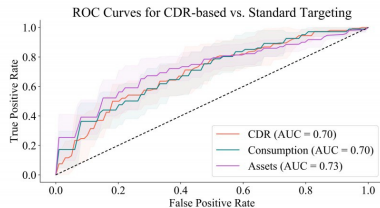
If the prediction itself is binary, the errors are summarised by a **confusion matrix**:

Pred. \ Act.	$y = 1$	$y = 0$
$\hat{y} = 1$	TP	FP (Type I)
$\hat{y} = 0$	FN (Type II)	TN

Misclassification loss:

$$\ell = \mathbb{I}[\hat{y} \neq y].$$

- ▶ **TP** true positive, **FP** false positive;
- ▶ **FN** false negative (Type II error), **TN** true negative;
- ▶ **ROC** curve: TPR vs. FPR over all thresholds; the **AUC** summarises overall ranking ability.



ROC curve source: Aiken et al. (2020)

Standard regression solution

Goal: small $E(\hat{f}(x) - y)^2$

E.g. use linear functions $\hat{f}(x) = \hat{\beta}'x = \hat{\beta}_0 + \sum_{j=1}^k \hat{\beta}_j x_j$

- From training data, pick the β that provides best in-sample fit:

$$\min_{\hat{\beta}} E(y - \hat{\beta}'x)^2 \rightarrow \min_{\hat{\beta}} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\beta}'x_i)^2$$

- Which optimality properties does OLS have?
- Is this optimal for prediction?

Illustration of the least-squares solution: the fitted hyperplane / coefficients are obtained by projecting y onto the column space of $\mathbf{X}$ (geometry of OLS).

Bias–variance decomposition

- Loss at new point $y = \beta'x + \epsilon$ ($E[\epsilon|x] = 0$):

$$(\hat{y} - y)^2 = (\hat{\beta}'x - \beta'x - \epsilon)^2$$

- Average over draws of training sample (and ϵ):

$$\begin{aligned} E_{T,\epsilon}[(\hat{y} - y)^2] &= E_T[(\hat{\beta}'x - \beta'x)^2] + E_\epsilon[\epsilon^2] \\ &= \underbrace{\left((E_T[\hat{\beta}] - \beta)'x \right)^2}_{\substack{\text{bias} \\ \text{approximation}}} + \underbrace{x'V_T(\hat{\beta})x}_{\substack{\text{variance} \\ \text{overfit}}} + \underbrace{V_\epsilon(\epsilon|x)}_{\text{irreducible noise}} \end{aligned}$$

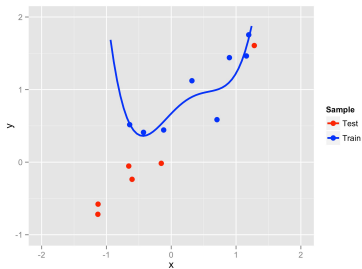
- Important framing within econometrics, stats

Expected prediction error

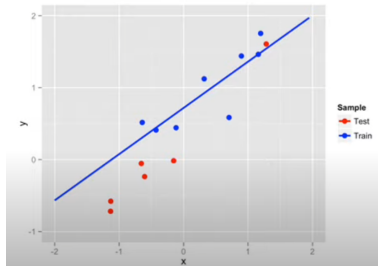
$$\mathbb{E}[(y - f(\mathbf{x}))^2] = \underbrace{\text{Bias}^2}_{\text{model error}} + \underbrace{\text{Variance}}_{\text{estimation error}} + \underbrace{\sigma^2}_{\text{noise}}$$

Approximation–overfit trade-off

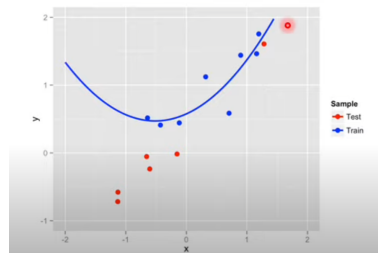
Three fits of increasing complexity on the same data:



too simple (underfit)



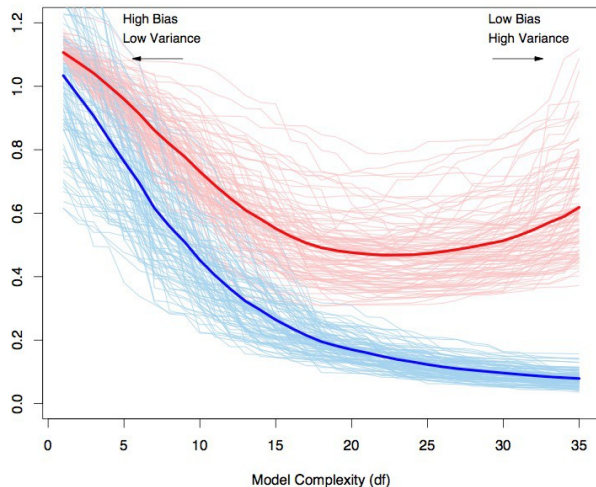
good complexity



too complex (overfit)

Left: high bias (misses the pattern); right: high variance (fits the noise); middle: best generalisation.

Approximation–overfit trade-off



- ▶ **In-sample fit** keeps improving as the model gets more flexible;
- ▶ **New-data fit** (test error) first improves, then degrades — the U-shaped test-error curve;
- ▶ Choose the complexity that minimises the *out-of-sample* error.

Source: Hastie, Tibshirani & Friedman (2009), *ESL*.

Approximation—overfit trade-off

As the model becomes more complex:

- ▶ It fits the **true function** better $\Rightarrow$ better approximation;
- ▶ It also fits the **noise** better $\Rightarrow$ overfitting.

Hence, in practice:

- ▶ Use flexible functional forms where needed;
- ▶ But **limit expressiveness** — this is exactly what **regularization (正则化)** does.

Regularization (正则化) for linear regression

- Rather than OLS

$$\min_{\hat{\beta}} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\beta}' x_i)^2$$

- Fit constrained problem

$$\min_{\hat{\beta}} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\beta}' x_i)^2 \text{ s.t. } \|\hat{\beta}\| \leq c$$

$$\|\hat{\beta}\|_0 = \sum_{j=1}^k 1_{\hat{\beta}_j \neq 0} \quad \|\hat{\beta}\|_1 = \sum_{j=1}^k |\hat{\beta}_j| \quad \|\hat{\beta}\|_2^2 = \sum_{j=1}^k \hat{\beta}_j^2$$

- Throughout, assume $\hat{\beta}' = [\hat{\beta}_0 \ \hat{\beta}_1, \dots, \hat{\beta}_k)$
- Normalize! not penalized

Ridge (ℓ_2 , left) shrinks all coefficients; LASSO (ℓ_1 , right) shrinks some coefficients exactly to zero, producing sparse models.

3. Supervised Machine Learning Algorithms: LASSO

In one popular type of penalized regression, **LASSO** (least absolute shrinkage and selection operator), the penalty term has the following form, with $\lambda > 0$:

LASSO objective

$$\hat{\beta} = \arg \min_{\beta} \left[\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^d \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^d |\beta_j| \right]$$

- ▶ Since LASSO eliminates the least important features from the model, it automatically performs **feature selection**;
- ▶ λ is a **hyperparameter** (set before learning): it balances fitting the model versus keeping the model parsimonious.

LASSO regression

$$\min_{\hat{\beta}} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\beta}' x_i)^2 + \lambda \sum_{j=1}^k |\hat{\beta}_j|$$

- ▶ **Selects and shrinks** (coefficients become exactly zero);
- ▶ “Capitalist” — in doubt, give all to one feature;
- ▶ Produces **sparse solutions**.

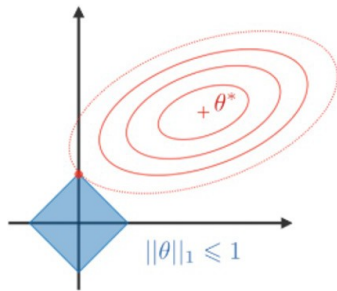
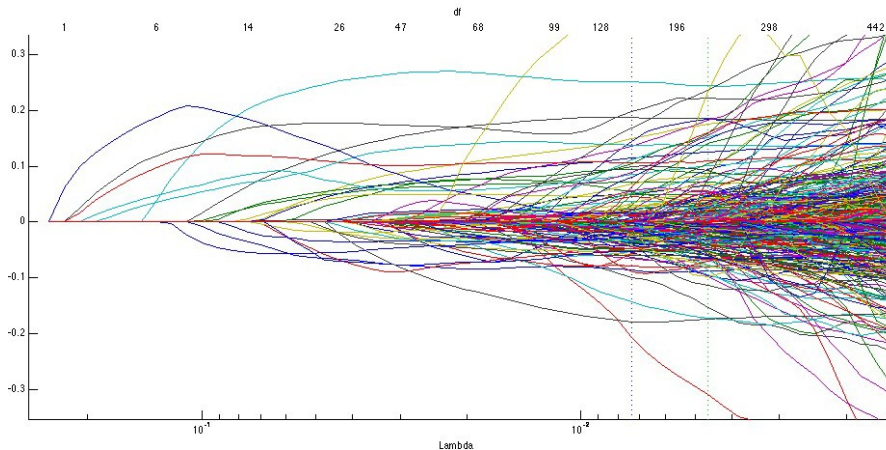


Illustration: Afshine Amidi and Shervine Amidi.

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 6 Logistic Regression and Linear Classifier

| Supervised Machine Learning Algorithms

LASSO regression: coefficient path



Coefficient sizes as λ varies Source: https://scikit-learn.org/stable/modules/linear_model.html

Ridge regression

$$\min_{\hat{\beta}} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\beta}' x_i)^2 + \lambda \sum_{j=1}^k \hat{\beta}_j^2$$

- ▶ Coefficients **shrink towards zero**, but **never quite** reach it;
- ▶ “Socialist” — in doubt, distribute weight to multiple features;
- ▶ Can be interpreted as a **Bayesian posterior** (Gaussian prior on $\mathbf{w}$).

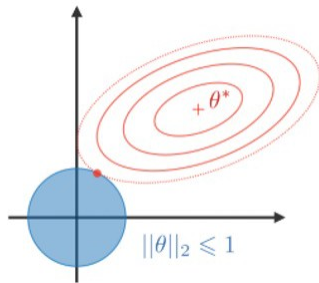
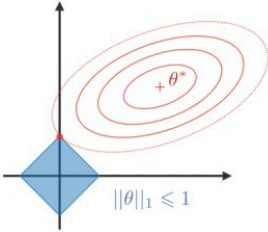
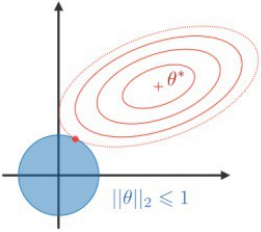
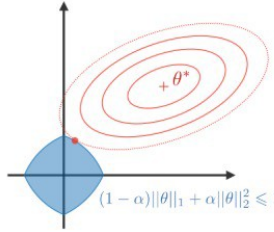


Illustration: Afshine Amidi and Shervine Amidi.

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 6 Logistic Regression and Linear Classifier

| Supervised Machine Learning Algorithms

Regularization for linear regression

LASSO	Ridge	Elastic Net
- Shrinks coefficients to 0 - Good for variable selection	Makes coefficients smaller	Tradeoff between variable selection and small coefficients
 A 2D plot showing red elliptical contours of the loss function centered at θ^*. A blue diamond-shaped region represents the L1 penalty constraint $\ \theta\ _1 \leq 1$. The red dot marks the point where the diamond is tangent to the innermost contour, which is on the positive x-axis.	 A 2D plot showing red elliptical contours of the loss function centered at θ^*. A blue circular region represents the L2 penalty constraint $\ \theta\ _2 \leq 1$. The red dot marks the point where the circle is tangent to the innermost contour.	 A 2D plot showing red elliptical contours of the loss function centered at θ^*. A blue rounded diamond-shaped region represents the Elastic Net constraint $(1-\alpha)\ \theta\ _1 + \alpha\ \theta\ _2^2 \leq 1$. The red dot marks the point where the boundary is tangent to the innermost contour.
$\dots + \lambda \ \theta\ _1$ $\lambda \in \mathbb{R}$	$\dots + \lambda \ \theta\ _2^2$ $\lambda \in \mathbb{R}$	$\dots + \lambda \left[(1-\alpha) \ \theta\ _1 + \alpha \ \theta\ _2^2 \right]$ $\lambda \in \mathbb{R}, \alpha \in [0, 1]$

Structure of supervised learners

Every supervised learner is built from three components:

1. A **function class** $\mathcal{H}$ (which functions f are allowed);
2. A **regularizer** (how complexity / size of f is penalised, encoding prior knowledge);
3. An **optimization algorithm** that gets us there (gradient descent, closed-form solution, ...).

Recap: linear regression vs. logistic regression

	Linear regression	Logistic regression
Function	$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$	$P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$
Loss	ℓ_2 (MSE)	negative log-likelihood
Derivation	MLE + Gaussian noise	MLE (Bernoulli)
Solution	closed form	gradient descent